

Integración de Servicios Cloud GCP

Linux, interfaces de red y troubleshooting en una máquina virtual

Semana 02 - Sesión 03

Rel Guzman



**Universidad
Tecnológica
del Perú**

- ▶ ¿Qué diferencia a un hipervisor tipo 1 de uno tipo 2?
- ▶ ¿Qué recursos del equipo físico consume una máquina virtual?
- ▶ ¿Por qué una máquina virtual es apropiada para un laboratorio local?
- ▶ ¿Qué representa una conexión entre dos componentes de una arquitectura?

Requisito indispensable

Debes tener instalado **Ubuntu Desktop** (Ubuntu con entorno gráfico). Puede ejecutarse directamente en el equipo o dentro de una máquina virtual creada con el hipervisor disponible.

Configuración mínima sugerida

Instalación nativa o VM funcional. Para una VM:
4 GB de RAM · 2 vCPU · 25 GB de disco · 1
adaptador con salida a internet.

Comprueba antes de clase

La VM inicia, permite ingresar al escritorio y abre Terminal desde el menú de aplicaciones. Conserva tu contraseña de `sudo`.

Logro de la sesión

Al finalizar, podrás reconocer Linux y su terminal, configurar **interfaces de red virtuales**, monitorear recursos del sistema y aplicar un método ordenado de **troubleshooting** para identificar fallas básicas.

- ▶ ¿Qué diferencia hay entre el equipo anfitrión y la máquina virtual?
- ▶ ¿Para qué sirve una dirección IP?
- ▶ ¿Qué recursos físicos comparte el hipervisor entre las VM?
- ▶ Cuando “no hay internet”, ¿qué revisarías primero?

¿Para qué te servirá lo aprendido?

Podrás preparar una VM Linux para un laboratorio, comprobar si tiene red y observar CPU, memoria, disco y procesos antes de proponer una solución.

Es la misma disciplina que usarás después con instancias de Compute Engine: observar el estado, reunir evidencia y corregir la causa sin adivinar.

Linux es el sistema; una distribución lo deja listo para usar

Unix

Familia de sistemas operativos creada para que varias personas y programas puedan utilizar un equipo de forma organizada. Introdujo ideas como usuarios, permisos, procesos y un único árbol de archivos.

Linux

Sistema operativo libre y de código abierto que sigue muchas ideas de Unix. Su kernel administra CPU, memoria, disco, red y dispositivos.

Distribución Linux

Sistema listo para instalar y usar. Reúne el kernel de Linux, programas, instalador, actualizaciones y herramientas de configuración. Ejemplos: Ubuntu, Debian, Mint y Kali.

Unix y Linux comparten ideas, pero no son lo mismo

Aspecto	Unix	Linux
Origen	Creado en 1969 en Bell Labs	Creado en 1991 por Linus Torvalds
¿Qué era originalmente?	Un sistema operativo completo	Un kernel
¿Qué es hoy?	Una familia de sistemas operativos	El kernel usado por muchas distribuciones
Código	Depende de cada versión; muchas son comerciales	Libre y de código abierto
Ejemplos	AIX, HP-UX y macOS	Ubuntu, Debian, Mint y Kali

Idea principal

Linux no es una versión de Unix. Es un kernel diferente que utiliza muchas de sus ideas y formas de trabajo.

Linux domina servidores porque se administra bien a distancia

Servidores

Estabilidad, bajo consumo, automatización y administración remota.

Nube

Muchas VM, contenedores y procesos de DevOps se ejecutan sobre Linux.

Operaciones

Logs, monitoreo, redes y servicios pueden revisarse desde la terminal.

En palabras simples

Aprender Linux no significa memorizar cientos de comandos: significa aprender a observar el sistema y pedirle acciones concretas.

Familia	Distribuciones conocidas	Gestor de paquetes
Debian	Debian, Ubuntu, Mint, Kali	apt y dpkg
Red Hat	RHEL, Fedora, Rocky, AlmaLinux	dnf y rpm
Arch / SUSE	Arch, Manjaro, openSUSE	pacman / zypper

Regla del laboratorio

Las instrucciones y paquetes se validarán en una distribución **basada en Debian**. La demostración se realizará en Ubuntu Desktop; Debian, Mint y Kali comparten la misma familia y utilizan herramientas como apt y dpkg.

Terminal Ventana o consola donde escribes y lees resultados.

Shell Programa que interpreta lo escrito; normalmente **Bash**.

Comando Instrucción que la shell ejecuta, con opciones y argumentos.

Prompt Línea que indica el usuario, equipo y ubicación antes de escribir.

Lee este ejemplo

```
ana@linux-lab:~$ ls -la /etc
```

Aquí `ls` es el comando, `-la` son opciones y `/etc` es el argumento.

Aspecto	Bash	Zsh
Qué es	Una shell muy usada en Linux	Otra shell compatible con muchos comandos de Bash
Uso habitual	Predeterminada en Ubuntu	Predeterminada en macOS moderno
Comandos	Ejecuta comandos de Linux	Ejecuta muchos de los mismos comandos

¿Qué shell tiene configurada tu cuenta?

```
echo $SHELL
```

El resultado suele ser `/bin/bash` o `/bin/zsh`. En Ubuntu normalmente verás `/bin/bash`.

Directorio	Uso principal
/	Raíz: inicio de todo el árbol de archivos
/home	Carpetas personales de los usuarios
/etc	Configuración del sistema y de servicios
/var	Datos que cambian, incluidos logs
/tmp	Archivos temporales
/usr/bin	Muchos comandos y programas instalados

Una ruta absoluta empieza con /; una ruta relativa empieza desde tu ubicación actual. Linux no usa unidades C: o D:.

Los primeros comandos responden cuatro preguntas

Pregunta	Comando	Qué debes observar
¿Quién soy?	whoami	Usuario de la sesión
¿En qué equipo estoy?	hostname	Nombre de la VM
¿Dónde estoy?	pwd	Ruta actual
¿Qué hay aquí?	ls -la	Archivos, propietarios y permisos
¿Qué Linux uso?	cat /etc/os-release	Distribución y versión

1. Inicia Ubuntu Desktop, de forma nativa o virtualizada, e ingresa con tu usuario.
2. Ejecuta: `whoami`, `hostname`, `pwd` y `cat /etc/os-release`.
3. Crea el directorio del curso: `mkdir -p ~/curso-cloud/sesion03`.
4. Ingresa: `cd ~/curso-cloud/sesion03` y confirma con `pwd`.
5. Guarda evidencia: `uname -a > evidencia.txt` y revísala con `cat evidencia.txt`.

Antes de copiar un comando

Lee qué hace, confirma la ruta y nunca uses `sudo` o `rm` solo porque apareció en internet.

Interfaz de red

Punto lógico mediante el cual el sistema envía y recibe tráfico. Puede representar un adaptador físico o uno virtual.

Dentro de Linux

Nombres habituales: `ens33`, `eth0`, `lo`. Cada interfaz tiene estado, direcciones y estadísticas.

En una máquina virtual

El hipervisor conecta el adaptador virtual a una red virtual. Linux lo detecta como si fuera una tarjeta de red real.

¿Cuántas interfaces puede tener una máquina virtual?

La cantidad no es siempre la misma

Depende del programa utilizado para crear la máquina virtual. Cada programa establece su propio límite.

En esta sesión

Empezaremos con **una interfaz de red**. Agregaremos otra solamente cuando necesitemos conectar la máquina a una segunda red.

Cinco comandos describen la red antes de probarla

Comando	Pregunta que responde
<code>ip -br link</code>	¿Qué interfaces existen y están UP o DOWN?
<code>ip -br addr</code>	¿Qué dirección tiene cada interfaz?
<code>ip route</code>	¿Por dónde sale el tráfico?
<code>resolvectl status</code>	¿Qué servidor DNS utiliza el sistema?
<code>ss -tulpn</code>	¿Qué puertos y servicios están escuchando?

Si `resolvectl` no está disponible, revisa `/etc/resolv.conf`.

1. Confirma tu shell con `echo $SHELL`.
2. Crea un alias: `alias listar='ls -la'`.
3. Ejecuta `listar` y observa el resultado.
4. Crea una función:
`info() { whoami; hostname; pwd; }`
5. Ejecuta `info`.

Alias

Nombre corto que representa un comando.

Función

Nombre que agrupa uno o varios comandos.

Duración

Lo creado estará disponible hasta que cierres la terminal. Más adelante aprenderás a guardarlo.

Monitoreo

Observación del estado y del consumo de recursos para detectar tendencias, límites o comportamientos anómalos.

Recursos principales

CPU · memoria · almacenamiento · procesos ·
red · servicios · logs.

Una fotografía y una película

`free -h` toma una fotografía. `top` actualiza
datos continuamente y deja ver la evolución.

Estos comandos forman un kit inicial de monitoreo

Recurso	Comandos	Qué observar
CPU y procesos	<code>top, ps aux</code>	carga, %CPU, proceso responsable
Memoria	<code>free -h, top</code>	disponible, usada, swap
Disco	<code>df -h, du -sh</code>	partición llena, carpeta grande
Red	<code>ip -s link, ss -tulpn</code>	errores, bytes, puertos
Servicios	<code>systemctl --failed</code>	unidades que fallaron
Logs	<code>journalctl -p err -b</code>	errores del arranque actual

Herramientas opcionales: `htop`, `iotop`, `vmstat`, `sar`. Primero domina las que suelen venir instaladas.

1. Abre la terminal de tu máquina con Ubuntu.
2. Ejecuta `uptime` y observa la carga del sistema.
3. Ejecuta `free -h` y revisa la memoria disponible.
4. Ejecuta `df -h` y revisa el espacio de la partición principal.
5. Ejecuta `ps aux --sort=-%cpu | head` e identifica el proceso con mayor uso de CPU.
6. Escribe una conclusión: “Observo..., por eso concluyo...”.

Una cifra aislada no siempre es un problema

Linux usa memoria libre como caché. En `free -h`, observa especialmente la columna **available**; luego relaciona el dato con síntomas y procesos.

Troubleshooting es un método para reducir incertidumbre

Troubleshooting

Proceso sistemático para identificar la causa de una falla, comprobarla con evidencia, aplicar una corrección y verificar el resultado.

Síntoma → alcance → hipótesis → prueba → cambio → verificación

La diferencia clave

Reiniciar puede ocultar un síntoma; diagnosticar explica la causa. Un cambio sin evidencia puede crear una segunda falla.

Caso

La máquina con Ubuntu no puede abrir páginas web. Debemos descubrir en qué punto se interrumpe la conexión.

1. ¿Existe y está activa? `ip -br link`
2. ¿Tiene dirección? `ip -br addr`
3. ¿Tiene ruta por defecto? `ip route`
4. ¿Llega al gateway? `ping -c 3 <gateway>`
5. ¿Llega a internet por IP? `ping -c 3 8.8.8.8`
6. ¿Resuelve nombres? `getent hosts google.com`
7. ¿El servicio responde? `curl -I https://example.com`

La primera prueba que falla delimita la capa donde conviene investigar.

Confusión	Aclaración
Linux y una distribución son lo mismo	Linux es el kernel; una distribución agrega programas y herramientas.
Terminal y shell son lo mismo	La terminal muestra la interacción; la shell interpreta comandos.
Un alias o una función quedan guardados	En la práctica actual desaparecen al cerrar la terminal.
Memoria “used” alta siempre es falla	Revisa available, swap, procesos y síntomas.
Troubleshooting es probar soluciones	Primero reúne evidencia y comprueba una posible causa.

1. **¿Cuántas interfaces?** Depende del programa utilizado para crear la máquina virtual. Cada programa establece su propio límite.
2. **¿Qué opciones de configuración existen?** Automática: la red entrega los datos. Manual: escribimos la dirección IP, la puerta de enlace y el DNS.
3. **¿Qué comandos monitorean Linux?** `uptime` para carga, `free -h` para memoria, `df -h` para disco y `top` o `ps` para procesos.
4. **¿Qué es troubleshooting?** Un proceso ordenado para observar una falla, reunir evidencia, comprobar una posible causa, corregirla y verificar el resultado.



gracias



Universidad
Tecnológica
del Perú

- ▶ Documentación del hipervisor o plataforma utilizada: límites de hardware virtual y modos de red disponibles.
- ▶ Debian Administrator's Handbook y páginas de manual: `ip`, `ss`, `free`, `top`, `df`, `systemctl`, `journalctl`.
- ▶ Material base proporcionado: *Fundamentos de Linux — Programa completo*, diapositivas iniciales reorganizadas para esta sesión.